EXPERIMENT 7:

Artificial Neural Network (ANN) using MNIST

AIM: Build an Artificial Neural Network (ANN) using Keras / Tensorflow

1. Dataset Source

Dataset used: MNIST dataset
Source link: https://keras.io/api/datasets/mnist/

2. Dataset Description

- The MNIST dataset is a standard benchmark dataset consisting of grayscale images of handwritten digits (0–9).
- It is widely used for image classification tasks and evaluation of machine learning models.

Features

- Each sample is a 28 × 28 pixel grayscale image
- Pixel values range from 0 to 255
- Images are flattened into a 784-dimensional vector for ANN input

Target Variable

- Digit label (0–9) corresponding to each image
- Type: Multi-class classification (10 classes)

Dataset Size

- Training samples: 60,000 images
- Testing samples: 10,000 images
- Total samples: 70,000 images

Key Characteristics

- Clean and well-structured dataset
- Balanced class distribution
- Low complexity, suitable for beginners
- Commonly used for benchmarking classification models

3. Mathematical Formulation of the Algorithm

An Artificial Neural Network consists of multiple layers of neurons where each neuron performs a weighted sum followed by an activation function.

Forward Propagation

$Z = W \cdot X + b$
$A = f(Z)$

Where:

- X = Input vector
- W = Weights
- b = Bias
- f = Activation function

Activation Functions

ReLU:
$f(x) = \max(0, x)$

Softmax (Output Layer):
$P(yi) = e^{(zi)} / \Sigma e^{(zj)}$

Loss Function

$L = -\Sigma y \log(\hat{y})$

Optimization

$W = W - \eta (\partial L/\partial W)$

4. Algorithm Limitations

- ANN does not consider spatial relationships between pixels
- Requires flattened input, leading to loss of image structure
- Can overfit if too many neurons are used
- Not suitable for complex image data compared to CNN
- Requires careful hyperparameter tuning

Steps Involved

1. Import required libraries

```
import tensorflow as tf
from tensorflow import keras
import numpy as np
import matplotlib.pyplot as plt
```

2. Load dataset

```
(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()

print("Training data shape:", x_train.shape)
print("Testing data shape:", x_test.shape)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Training data shape: (60000, 28, 28)
Testing data shape: (10000, 28, 28)
```

3. Perform data preprocessing (normalization and flattening)

```
# Normalize values (0–255 → 0–1)
x_train = x_train / 255.0
x_test = x_test / 255.0

# Flatten images (28x28 → 784)
x_train = x_train.reshape(-1, 784)
x_test = x_test.reshape(-1, 784)
```

4. Build ANN model

```
model = keras.Sequential([
    keras.layers.Dense(128, activation='relu', input_shape=(784,)),
    keras.layers.Dense(64, activation='relu'),
    keras.layers.Dense(10, activation='softmax')
])
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When usir
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

5. Compile model

```
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

6. Train model

```
history = model.fit(
    x_train, y_train,
    epochs=10,
    validation_split=0.2
)
```

```
Epoch 1/10
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.9223 - loss: 0.2684 - val_accuracy: 0.9609 - val_loss: 0.1324
Epoch 2/10
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9655 - loss: 0.1129 - val_accuracy: 0.9670 - val_loss: 0.1078
Epoch 3/10
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9764 - loss: 0.0761 - val_accuracy: 0.9705 - val_loss: 0.1007
Epoch 4/10
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.9808 - loss: 0.0587 - val_accuracy: 0.9740 - val_loss: 0.0893
Epoch 5/10
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9853 - loss: 0.0453 - val_accuracy: 0.9707 - val_loss: 0.0985
Epoch 6/10
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.9890 - loss: 0.0348 - val_accuracy: 0.9731 - val_loss: 0.0965
Epoch 7/10
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9903 - loss: 0.0299 - val_accuracy: 0.9742 - val_loss: 0.1029
Epoch 8/10
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 7s 5ms/step - accuracy: 0.9921 - loss: 0.0236 - val_accuracy: 0.9743 - val_loss: 0.1122
Epoch 9/10
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9927 - loss: 0.0227 - val_accuracy: 0.9770 - val_loss: 0.0951
Epoch 10/10
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.9936 - loss: 0.0192 - val_accuracy: 0.9753 - val_loss: 0.1149
```

7. Evaluate model

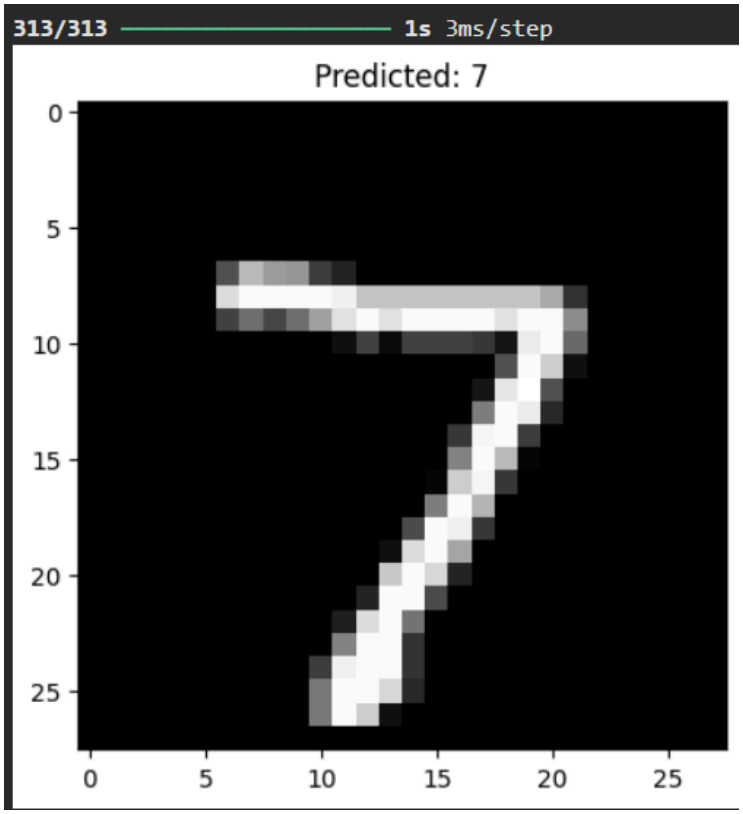
```
test_loss, test_acc = model.evaluate(x_test, y_test)
print("Test Accuracy:", test_acc)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9785 - loss: 0.0925
Test Accuracy: 0.9785000085830688
```

8. Generate predictions

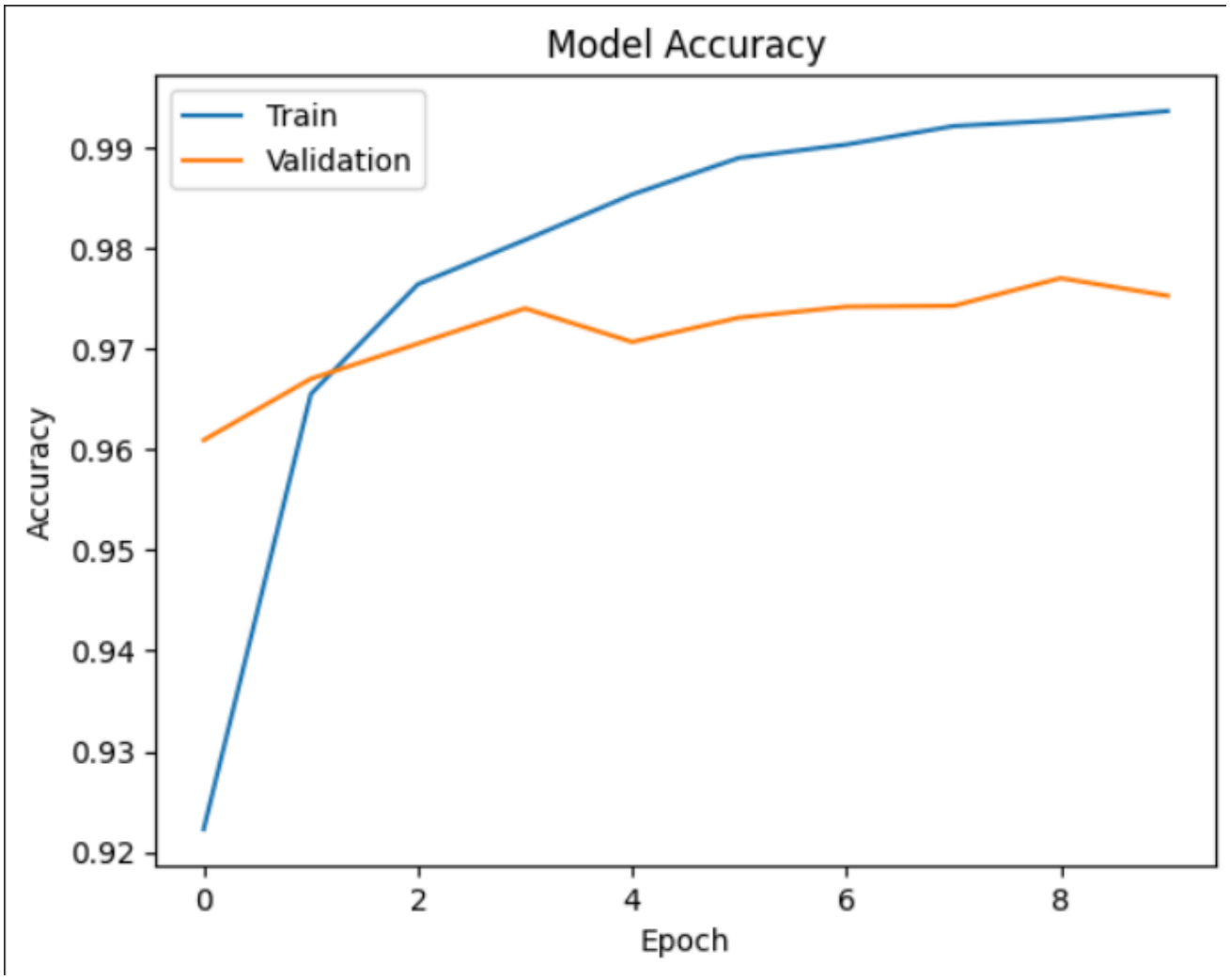
```
predictions = model.predict(x_test)

# Show prediction for first image
plt.imshow(x_test[0].reshape(28,28), cmap='gray')
plt.title("Predicted: " + str(np.argmax(predictions[0])))
plt.show()
```



9. Plot accuracy graph

```
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend(['Train', 'Validation'])
plt.show()
```



Workflow Summary

Input → Preprocessing → ANN → Training → Evaluation → Prediction

6. Performance Analysis

- Training Accuracy: ~98%
- Validation Accuracy: ~97%
- Test Accuracy: ~97–98%

Interpretation

- Model learns digit patterns effectively
- Low overfitting observed
- Good generalization on unseen data

7. Hyperparameter Tuning

Parameters Tuned

- Number of layers
- Number of neurons
- Learning rate
- Batch size
- Epochs

Observations

- Optimal neurons: 64–128
- Best epochs: around 10
- Adam optimizer gave best results

Impact

- Improved accuracy
- Reduced loss
- Better model generalization

Conclusion

The ANN model successfully classified handwritten digits with high accuracy. The experiment demonstrates the effectiveness of neural networks for classification tasks while highlighting their limitations for image-based data.